DBT Day 6

Shivam Zanjurne · Wed, 14 Oct 26 · Indexes (B-tree / B+ tree, clustered vs secondary, UNIQUE / composite / FULLTEXT, EXPLAIN speed demo) · Stored programs overview · Stored procedures (DELIMITER, variables, IN / OUT / INOUT, loops)

Table of Contents

1. What Is an Index?
2. Benefits and Costs of Indexes
3. How InnoDB Stores Indexes — B-tree and B+ tree
4. Clustered Index (Primary Key Index)
5. Setup: emp_demo and Looking at Indexes
6. Secondary Indexes — Unique, Single-Column, Composite
7. FULLTEXT Index
8. Clustered vs Secondary — What Lives on the Leaf?
9. Dropping Indexes
10. EXPLAIN Demo — Seeing Index Speed
11. Index Design Tips from Class
12. Types of Stored Programs in MySQL
13. Why Use Stored Procedures? Benefits and Drawbacks
14. DELIMITER — The Most Important "Gotcha"
15. Your First Stored Procedure
16. Variables: Local DECLARE vs Session @variables
17. SET and SELECT ... INTO
18. Parameters: IN, OUT, INOUT
19. Loops — Bulk Insert with WHILE
20. Inspecting Procedures
21. Day Wrap and Practice Checklist

What Is an Index?

Plain-English idea

An **index** is an extra structure MySQL builds to find rows **faster**, similar to the index at the back of a textbook:

- Without an index, finding "employee named Rahul" can mean scanning **every row** (a *table scan*).
- With an index on `emp_name`, MySQL can jump near the matching names using a sorted tree, then fetch only the needed rows.

Indexes do **not** replace tables. They are **helpers** that point toward data (or, in the special clustered case, *are* how the table data is ordered).

Mental model

```
Query:  WHERE emp_name = 'Rahul'

Without index:  check row1, row2, row3, … row 100000   → slow
With index:     tree search → leaf with key 'Rahul' → go to that row  → fast
```

Who creates indexes?

Situation	What happens
You declare a **PRIMARY KEY**	InnoDB **automatically** creates a **clustered index**
You declare a **FOREIGN KEY**	MySQL typically creates an index on the FK column (class showed this in `SHOW INDEX`)
You declare **UNIQUE**	An index is created to enforce uniqueness
You run `CREATE INDEX` / `CREATE UNIQUE INDEX` / `CREATE FULLTEXT INDEX`	You add a **secondary** (or fulltext) index yourself

---

Benefits and Costs of Indexes

Class started with theory: indexes help, but they are not free.

Benefits

1. **Faster filtering, lookups, and joins** when the index **matches** the query
   Example: `WHERE emp_name = '…'`, `JOIN … ON a.id = b.id` if `id` is indexed.
2. **Can help** `ORDER BY` **/** `GROUP BY` in suitable cases
   If data is already ordered the way the index is ordered, MySQL may avoid an extra sort.

Costs / downsides

1. **Extra storage** — indexes take disk (and memory when cached).
2. **Slower writes** — every `INSERT` / `UPDATE` / `DELETE` must also update relevant indexes.
3. **Maintenance overhead** — too many indexes or indexes on the wrong columns can **slow** the system instead of helping.

> Class warning: if the table is **small**, or a column is **rarely searched**, do **not** create indexes "just because." Types are for learning; real design is selective.

How InnoDB Stores Indexes — B-tree and B+ tree

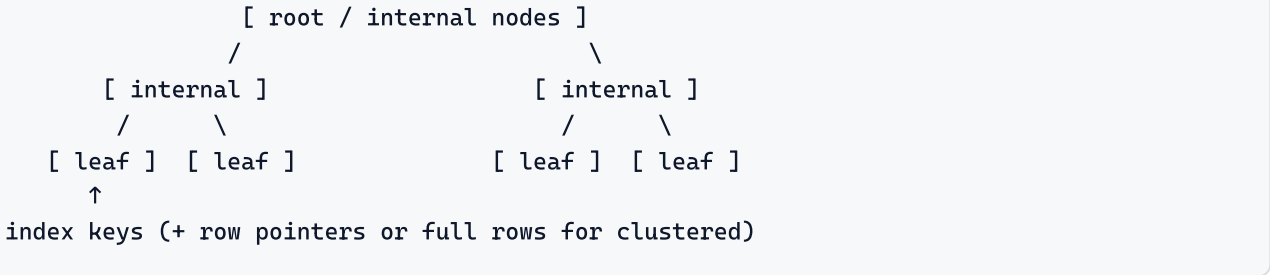
InnoDB is the storage engine

MySQL talks SQL; **InnoDB** decides how rows and indexes sit on disk/pages. Day 5 already mentioned pages (~16 KB) and the buffer pool. Indexes live in that world too.

B-tree = balanced / self-balancing tree

- A **B-tree** (balanced tree) keeps itself balanced when nodes are added.
- Important property class stressed: **all leaf nodes stay at the same level** — search depth stays controlled.
- Values in the tree are kept in **sorted order**, so binary-search-like navigation works.

Where are index values stored?



Node type	Role
Internal nodes	Guide the search ("go left / go right") toward the correct leaf
Leaf nodes	Hold the actual **index values** (and, depending on index type, row data or primary-key pointers)

B-tree vs B+ tree (exam nuance)

- When you run `SHOW INDEX`, MySQL reports index type `BTREE`.
- Class said: **internally InnoDB uses a B+ tree** flavour for indexes.
- For exams: say **InnoDB index type is B-tree (B+ tree internally)**; leaves hold the searchable keys.

Why a tree beats scanning

Example search path idea (same spirit as class drawing):

```
Looking for key 12 in a sorted tree:
  root splits ranges → go toward 25 → then toward 12 → land on leaf
Few hops instead of reading 100,000 unordered rows.
```

Clustered Index (Primary Key Index)

Primary key properties (recap)

A **PRIMARY KEY** is:

- **UNIQUE** (no duplicates)
- **NOT NULL** (no NULLs)

What InnoDB does automatically

When you create a table **with a primary key**, InnoDB creates a **clustered index** on that PK.

> **Clustered index** = the index that **defines how table rows are physically organized** (by primary key order) inside InnoDB's pages.

Critical rules

Rule	Meaning
Only one clustered index per table	Rows can only be stored in **one** physical order
Clustered index is usually the **PRIMARY KEY**	PK becomes the "main" index
If you **don't** define a PK	InnoDB picks a suitable `NOT NULL` + `UNIQUE` column if possible; otherwise it creates a **hidden clustered index**

Key–value picture (class example)

On a leaf for clustered index, think:

```
Key (PK) = 1
Values   = Aniket, 90000, department 10, …   (the whole row lives with the key)
```

Secondary indexes work differently (next sections): their leaves store **indexed columns + PK**, then use the PK to find the full row in the clustered index.

Setup: emp_demo and Looking at Indexes

Class created a demo employee table:

```sql
CREATE TABLE emp_demo (
    emp_id     INT PRIMARY KEY,
    emp_name   VARCHAR(30),
    department VARCHAR(30),
    salary     INT
);
```

DESCRIBE vs SHOW INDEX

```sql
DESCRIBE emp_demo;
-- shows columns, null, key, default, extra

SHOW INDEX FROM emp_demo;
-- shows every index: name, column(s), uniqueness, nullable, index type (BTREE), …
```

What you should notice for the PK index:

Observation	Meaning
Key / index on `emp_id`	Clustered / primary index exists
`Null` column empty / not YES for PK	NULLs not allowed
`Index_type` = `BTREE`	Tree-based index
Cardinality	Rough distinct-key estimate (0 on empty table is normal)

Foreign keys also show up in SHOW INDEX

On a real `employee` table with `department_id` FK, `SHOW INDEX FROM employee` often shows:

- an index for the **PRIMARY KEY**
- an index for the **FOREIGN KEY** column

For the FK index, `Null = YES` can appear because FK columns may allow NULL (employee not yet assigned a department).

Secondary Indexes — Unique, Single-Column, Composite

Anything you create **besides** the clustered PK index is a **secondary index**.

1) UNIQUE index

Enforces uniqueness (like UNIQUE constraint) and speeds lookups on that column.

```
-- email may be NULL (UNIQUE allows multiple NULLs in MySQL),
-- but non-NULL values must be unique
CREATE UNIQUE INDEX idx_emp_email
ON emp_demo (email_id);
```

```
SHOW INDEX FROM emp_demo;
-- you should see idx_emp_email, Null=YES possible, Index_type=BTREE
```

> Class also wrote `CREATE INDEX …` naming patterns like `emp_email_unique`. Name indexes clearly so `DROP INDEX` is easy later.

2) Single-column (non-unique) index

Salary can repeat — still useful to index if you often filter/sort by salary.

```
CREATE INDEX idx_emp_salary
ON emp_demo (salary);

SHOW INDEX FROM emp_demo;
```

3) Composite (multi-column) index

One index spanning **two or more columns**:

```
CREATE INDEX idx_emp_id_salary
ON emp_demo (emp_id, salary);

-- another class demo shape:
CREATE INDEX idx_dept_name_salary
ON emp_demo (department, emp_name, salary);
```

How to think about composite indexes:

- Useful when queries filter/join/sort on **that combination** (or a **leftmost prefix** of the column list).
- Example: index on `(department, emp_name, salary)` helps:
- `WHERE department = …`
- `WHERE department = … AND emp_name = …`
- `WHERE department = … AND emp_name = … AND salary = …`
- It does **not** magically help a query that only filters `salary` alone (rightmost column only) — the left side of the index matters.

FULLTEXT Index

Purpose

Ordinary B-tree indexes are great for equality / range on short keys (`emp_id` , `salary` , `name = 'X'`).

FULLTEXT indexes support **word / term search** inside longer character/text content (articles, book content, descriptions) — not the same as `WHERE emp_id = 5` .

Syntax class used

```
CREATE TABLE books (
    book_id INT PRIMARY KEY,
    title   VARCHAR(100),
    content TEXT
);

CREATE FULLTEXT INDEX idx_books_content
ON books (content);

SHOW INDEX FROM books;
```

Notes from class:

- For `TEXT` / long character searching, use **FULLTEXT** (write `FULLTEXT` as one idea: “full text”).
- Prefer creating a **PRIMARY KEY first**, then add text columns / fulltext indexes — class hit an error creating an index when key/length setup was wrong without a proper PK path.

Clustered vs Secondary — What Lives on the Leaf?

This is the theoretical heart of Day 6 indexes.

	Clustered index	**Secondary index**
How many per table?	**Exactly one**	Many allowed
Usually on	PRIMARY KEY	Any chosen column(s)
Leaf contains	Index key + **full row data** (row organized by PK)	Indexed column(s) + **PRIMARY KEY value** (pointer into clustered index)
Created by	InnoDB automatically with PK (or hidden PK)	`CREATE INDEX` / UNIQUE / FULLTEXT / FK helpers

Lookup with a secondary index (two steps)

```
1) Search secondary index tree for emp_name = 'Rahul'
   → leaf gives primary key value(s), e.g. emp_id = 587

2) Use emp_id = 587 in the clustered index
   → fetch the full row
```

That is why secondary indexes still need the PK: they don't store the whole row (normally); they store enough to **locate** the row.

Dropping Indexes

```sql
DROP INDEX idx_emp_id_salary ON emp_demo;

SHOW INDEX FROM emp_demo;
-- the dropped index should be gone; PK clustered index remains
```

You cannot casually "drop the clustered index" while keeping the same PK model — the PK *is* the clustered organization. Drop secondary indexes you no longer need to reduce write cost.

EXPLAIN Demo — Seeing Index Speed

Class loaded many rows into `emp_demo` (via a stored procedure — see later), then compared search **with vs without** an index using `EXPLAIN`.

What EXPLAIN does

`EXPLAIN` shows the **optimizer's plan**: will it use an index? how many rows might it touch? what access type?

```sql
EXPLAIN
SELECT *
FROM emp_demo
WHERE emp_name = 'Employee_587';    -- example name pattern from class CONCAT inserts
```

Before creating an index on emp_name (typical observation)

EXPLAIN field	Rough meaning when bad
`possible_keys`	**NULL** — no candidate index
`key`	NULL — none chosen
`type`	often `ALL` — full table scan
`rows`	large estimate (many rows examined)

After

```
CREATE INDEX idx_emp_name
ON emp_demo (emp_name);

EXPLAIN
SELECT *
FROM emp_demo
WHERE emp_name = 'Employee_587';
```

EXPLAIN field	Rough meaning when good
`possible_keys`	`idx_emp_name` appears
`key`	`idx_emp_name` chosen
`type`	often `ref` (index lookup) instead of `ALL`
`rows`	can drop dramatically (class saw **1** for a selective match)
`key_len`	length of index portion used (class mentioned ~203 for a varchar name index)

How to practice the demo yourself

1. Insert a large number of rows (procedure below).
2. `EXPLAIN` a name search — note `type` / `rows` / `possible_keys`.
3. `CREATE INDEX` on that column.
4. `EXPLAIN` again — confirm the index is used.

That difference is the whole point of indexes in one screenshot.

Index Design Tips from Class

1. **PK → clustered index** comes free — design a good PK.

2. Index columns you **filter, join, or sort** on often.
3. Prefer **composite** indexes that match real `WHERE` column order (leftmost prefix).
4. Use **FULLTEXT** for word search in long text, not as a substitute for normal B-tree equality indexes.
5. **Don't spray indexes** on every column of a small table — writes and storage suffer.
6. Use `SHOW INDEX` and `EXPLAIN` — don't guess.

Types of Stored Programs in MySQL

Class listed **four** stored program types:

Type	Purpose (class wording)	How it runs
Stored procedure	Perform a **task** / group of SQL operations	Explicitly with `CALL`
Stored function	**Calculate and return a value**	Used **inside expressions / SQL** like a function
Trigger	**Respond automatically** to a table event (INSERT/UPDATE/DELETE)	Fires by itself when the event happens
Event	Run a **scheduled** database task (like a cron inside MySQL)	Scheduler runs it on a timetable

Tiny examples of intent

- **Procedure:** "Insert 1 lakh demo employees", "apply business rules for payroll batch".
- **Function:** "Return grade letter for marks".
- **Trigger:** "When a student is deleted, also write an audit row".
- **Event:** "Every night take a backup / purge old logs" (class mentioned scheduled tasks).

> Day 6 went deep on **procedures**. Functions / triggers / events were introduced as theory for later practice.

Why Use Stored Procedures? Benefits and Drawbacks

Benefits / reasons to use (class)

1. **Reusability** — write once, `CALL` many times; unlike a temporary table, the procedure stays until you drop it.

2. **Reduced network traffic** — one `CALL` can replace many round-trips of separate SQL statements from the client.
3. **Central business rules** — logic lives in the database; every app uses the same procedure.
4. **Permission control** — grant `EXECUTE` on a procedure without granting raw table rights for every step.
5. **Simpler for end users / apps** — they call one name instead of remembering a long script.

Drawbacks (class)

1. **Database-specific** — MySQL procedure syntax may not port cleanly to PostgreSQL / other engines; migration gets harder.
2. **Poorly written procedures can still be slow** — a procedure is not magic performance; bad SQL inside is still bad SQL.

DELIMITER — The Most Important "Gotcha"

The problem

Normally MySQL client treats `;` as "end of statement."

Inside a procedure you need **many** semicolons:

```
BEGIN
    DECLARE i INT DEFAULT 1;
    WHILE i ≤ 10 DO
        INSERT INTO …;
        SET i = i + 1;
    END WHILE;
END
```

If `;` still means "send to server now", the client would send a broken half-procedure after the first `;`.

The fix

Temporarily change the statement end marker:

```
DELIMITER $$

CREATE PROCEDURE show_student()
BEGIN
    SELECT * FROM student;
    SELECT COUNT(*) FROM student;
END $$

DELIMITER ;
```

Step	Meaning
`DELIMITER $$`	From now on, only `$$` ends a client statement (`;` is plain text inside)
`… END $$`	Send the whole `CREATE PROCEDURE` body
`DELIMITER ;`	**Restore** normal `;` behaviour

Class failure demo (remember this)

If you forget the final `DELIMITER ;`:

- The procedure might be created, but the client is still waiting for `$$`.
- Later `SELECT …;`, `EXIT`, etc. appear to “do nothing” / hang.
- Fix: run `DELIMITER ;` or reconnect and be careful next time.

> Also: put a **space** correctly: `DELIMITER ;` (keyword, space, semicolon). Class debugging called this out.

Your First Stored Procedure

Skeleton

```
DELIMITER $$

CREATE PROCEDURE show_student()
BEGIN
    -- any number of SQL statements
    SELECT * FROM student;
    SELECT COUNT(*) AS total_students FROM student;
END $$

DELIMITER ;
```

Run it

```
CALL show_student();
```

You should see result sets from both `SELECT`s.

Minimal “calculate” demo from class

```
DELIMITER $$

CREATE PROCEDURE calculate_total()
BEGIN
    DECLARE total INT DEFAULT 0;
    SET total = 100 + 200;
    SELECT total AS total_value;
END $$

DELIMITER ;

CALL calculate_total();
-- total_value = 300
```

Variables: Local DECLARE vs Session @variables

Local variables

Declared **inside** `BEGIN … END` with `DECLARE`. Scope = that procedure invocation.

```
DELIMITER $$

CREATE PROCEDURE variable_demo()
BEGIN
    DECLARE x INT DEFAULT 10;   -- local
    SELECT x AS local_value;
END $$

DELIMITER ;
```

Rules of thumb:

- `DECLARE` must appear at the **start** of the `BEGIN` block (before executable statements), in standard procedure style.
- Local `x` is **not** automatically visible after the procedure ends.

Session (user-defined) variables

Use `@name`. They live for the **session** (connection).

```
SET @x = 10;
SELECT @x AS session_value;
```

Class contrasted:

```
DELIMITER $$

CREATE PROCEDURE variable_demo()
BEGIN
    DECLARE x INT DEFAULT 5;          -- local
    SET @x = 99;                      -- session variable
    SELECT x AS local_student, @x AS session_value;
END $$

DELIMITER ;

CALL variable_demo();
SELECT @x;   -- still 99 after CALL (session scope)
```

Kind	Syntax	Lifetime
Local	`DECLARE x INT`	Current procedure run
Session	`@x`	Until disconnect / overwritten

SET and SELECT ... INTO

SET — assign an expression

```
SET total = 100 + 200;
SET @num = 10;
```

SELECT ... INTO — pull a query result into a variable

Classic pattern from class: read one student's marks into a local variable.

```sql
-- assume student(student_id, student_name, marks)

DELIMITER $$

CREATE PROCEDURE get_student_marks(IN sid INT)
BEGIN
    DECLARE v_marks DECIMAL(5, 2);

    SELECT marks
    INTO v_marks
    FROM student
    WHERE student_id = sid;

    SELECT v_marks AS marks_for_student;
END $$

DELIMITER ;

CALL get_student_marks(101);
-- e.g. Amit → 85.50
```

Reading the flow:

1. `IN sid` receives the student id you pass in `CALL`.
2. `SELECT marks INTO v_marks` stores **one** value into the local variable (query should return one row).
3. Final `SELECT` displays it to the caller.

If multiple rows match, `SELECT … INTO` errors — keep the `WHERE` selective (usually by PK).

Parameters: IN, OUT, INOUT

Procedures can take parameters with a **mode**:

Mode	Direction	Meaning
IN	Caller → procedure	Input only (default idea: pass a value in)
OUT	Procedure → caller	Procedure **writes** a result into a variable you supply
INOUT	Both	Pass a value in; procedure may change it; you read it after

IN — already seen

```sql
CREATE PROCEDURE get_student_marks(IN sid INT)
 …
CALL get_student_marks(101);
```

OUT — return a count

```
DELIMITER $$

CREATE PROCEDURE get_student_count(OUT s_count INT)
BEGIN
    SELECT COUNT(*)
    INTO s_count
    FROM student;
END $$

DELIMITER ;

CALL get_student_count(@total_students);
SELECT @total_students AS total_students;
```

Why `@total_students`?

`OUT` parameters are filled into **user variables** (or similar caller-side variables) at `CALL` time. You don't pass a literal number; you pass a variable that will receive the output.

INOUT — modify and return

Class demo: double a number.

```
DELIMITER $$

CREATE PROCEDURE double_num(INOUT num INT)
BEGIN
    SET num = num * 2;
END $$

DELIMITER ;

SET @num = 10;
CALL double_num(@num);
SELECT @num AS doubled;    -- 20
```

Story:

1. `@num` starts at 10 (session variable).
2. `CALL double_num(@num)` passes it as INOUT.
3. Procedure sets `num = num * 2`.
4. After CALL, `@num` is 20.

Loops — Bulk Insert with WHILE

Class used a procedure to insert **thousands / lakhs** of rows into `emp_demo` so the index `EXPLAIN` demo was realistic.

Pattern

```
DELIMITER $$

CREATE PROCEDURE insert_into_employee()
BEGIN
    DECLARE i INT DEFAULT 1;

    WHILE i ≤ 100000 DO
        INSERT INTO emp_demo (emp_id, emp_name, department, salary)
        VALUES (
            i,
            CONCAT('Employee_', i),
            CONCAT('Dept_', (i MOD 10)),
            30000 + (i MOD 50) * 1000
        );
        SET i = i + 1;
    END WHILE;
END $$

DELIMITER ;

CALL insert_into_employee();
-- takes time; wait for completion
```

Piece	Role
`DECLARE i INT DEFAULT 1`	Loop counter
`WHILE i ≤ N DO … END WHILE`	Repeat body
`CONCAT('Employee_', i)`	Build a searchable name
`SET i = i + 1`	Avoid infinite loop

Class operational notes

- Calling with a huge `N` takes noticeable time — don't abort mid-way unless you intend to.
- If aborted, you may have a **partial** load (e.g. tens of thousands of rows). Then:

```
DELETE FROM emp_demo;          -- or TRUNCATE (DDL — remember implicit commit)
CALL insert_into_employee(); -- try again
```

- Change `100000` to `1000` while learning; raise it when you want a serious `EXPLAIN` contrast.

Inspecting Procedures

```
SHOW PROCEDURE STATUS
WHERE Db = 'october_14';    -- your database name

-- or see routines in information_schema, etc.
```

Class listed procedures such as `calculate_total`, `get_student_marks`, `variable_demo`, `get_student_count`, `insert_into_employee`.

Drop when experimenting:

```
DROP PROCEDURE IF EXISTS insert_into_employee;
```

Day Wrap and Practice Checklist

Indexes — must remember

Topic	One-liner
Index	Extra structure for faster find/join/sort
Benefit	Faster matched lookups; can help ORDER BY/GROUP BY
Cost	Storage + slower writes; bad indexes hurt
Structure	B-tree (B+ internally); keys on leaves
Clustered	One per table; usually PK; stores rows by PK
Secondary	Many; leaf = indexed cols + PK pointer
FULLTEXT	Word search on text
Tools	`SHOW INDEX`, `CREATE/DROP INDEX`, `EXPLAIN`

Stored procedures — must remember

Topic	One-liner
Four programs	Procedure, function, trigger, event
Call procedure	`CALL name(…);`
DELIMITER	Change client end-marker; always restore `;`
Local vs session	`DECLARE x` vs `@x`
Assign	`SET` / `SELECT … INTO`
Params	`IN` / `OUT` / `INOUT`
Loop	`WHILE … DO … END WHILE`

Practice (lab order)

1. Create `emp_demo` with PK; `SHOW INDEX`; confirm BTREE clustered index.
2. Add UNIQUE / single / composite indexes; `SHOW INDEX` after each.
3. Create a `books` table + `FULLTEXT` index.
4. Load ~10k–100k rows with a `WHILE` procedure.
5. `EXPLAIN` name search before and after `CREATE INDEX` on `emp_name`.
6. Write procedures for: display + count; `SELECT INTO`; `OUT` count; `INOUT` double.
7. Intentionally forget `DELIMITER ;` once — see the hang — then fix it (so you never forget in the exam).

> Coverage: DBT Day 6 lecture (14 Oct 2026) — expanded from the Hinglish transcript into step-by-step English notes with runnable MySQL patterns. Triggers/functions/events were introduced; deep dive was indexes + procedures.